From-Scratch Model Quantization in PyTorch

Theory, implementation flow, experiments, results, and conclusion

Contents

1 Executive summary 1
2 Project scope and evaluation question 1
3 Theory and core concepts 1
3.1 Why quantize weights? 1
3.2 Packing and metadata overhead 2
3.3 Error and quality metrics 2
3.4 Clipping search 2
4 Project structure and implementation flow 3
4.1 Main components 3
4.2 End-to-end flow 3
4.3 Sanity checks recorded in the notebook 3
5 Experimental design 4
6 Results from `results.csv` 4
6.1 Size and quality table 4
6.2 Weight error and runtime table 5
6.3 Figures from the notebook 5
7 Interpretation and project conclusion 8
7.1 Scope of the evidence 8
8 Glossary 9
9 Project links 9

1 Executive summary

This project implements weight-only post-training quantization for the Qwen/Qwen2.5-0.5B language model using plain PyTorch tensor operations. It compares seven configurations that vary bit width, scaling granularity, symmetry, and clipping. The supplied notebook includes sanity checks, a WikiText-2 evaluation, decoding-speed and GPU-memory measurements, charts, CSV export, and a save/reload check for a selected configuration.

The project sets two simultaneous targets: reduce model tensor storage by at least 60% relative to FP16 and keep WikiText-2 perplexity within 15% of the FP16 baseline. The baseline is 942.3 MB and has perplexity 17.19, so the size limit is approximately 376.9 MB and the quality limit is approximately 19.77. No tested configuration satisfies both targets. The notebook's fallback choice is W4 group-128: it is 308.9 MB (67.2% smaller) with perplexity 20.65 (+20.1%), so it meets the size target but misses the quality budget. Its export reload check passes at a maximum logit difference of 5.86e-2 against the notebook's 6e-2 threshold.

The most important measured pattern is that granularity materially changes quality at the same bit width. The W4 per-tensor model has perplexity 10,446,818.97, while W4 per-channel reduces this to 30.79 and W4 group-128 to 20.65. Conversely, the two W8 configurations preserve the baseline quality budget but do not reach the required 60% size reduction. All quantized configurations decode more slowly than FP16 in the recorded plain-PyTorch implementation, which reconstructs weights during each linear-layer forward pass.

2 Project scope and evaluation question

The project asks whether a small-from-scratch quantizer can reduce the tensor storage of a 494-million-parameter causal language model while retaining language-model quality. It specifically explores whether changing how weights share scales—one scale for a whole tensor, one per output channel, or one per group—can make lower-bit formats practical under the notebook's chosen quality and size constraints.

The source notebook is the implementation and experiment record. The supplied `results.csv` contains the FP16 baseline and seven configuration-level results. `layer_sqnr.csv` contains per-layer reconstruction metrics for seven configurations. All numerical findings below are taken from those supplied artifacts and notebook outputs; the report does not claim a new model run.

3 Theory and core concepts

3.1 Why quantize weights?

A floating-point weight normally occupies more storage than a small integer code. Quantization replaces each weight with a compact integer level and stores metadata so an approximate floating-point value can be recovered. A lower bit width allows more values to be packed into each byte, but it also reduces the number of representable levels. The project measures the consequences as both stored model size and the resulting change in evaluation perplexity.

In an asymmetric scheme, the notebook derives a scale from a local minimum and maximum, computes an integer offset, clamps and rounds each value to the representable range, and reconstructs an approximation:

$$s = \frac{x_{\max} - x_{\min}}{2^b - 1}$$
$$z = \mathrm{round}\left(-\frac{x_{\min}}{s}\right), \quad q = \mathrm{clamp}\left(\mathrm{round}\left(\frac{w}{s}\right) + z, 0, 2^b - 1\right)$$
$$\hat{w} = (q - z)s$$

Here, b is the bit width, q is the stored integer, s is the scale, and z is the zero-point. The reconstruction error is the difference between the original weight and $\hat{w}$. For symmetric quantization, the representation is centered around zero with a fixed integer offset; the notebook stores no per-row zero-point for that case. The quantizer's granularity determines which weights share the same scale. Per-tensor scaling uses one range for an entire matrix. Per-channel scaling uses one range for each output row. Group-wise scaling divides a row into fixed-size groups, such as 128 weights, and quantizes each group separately. Smaller sharing regions can limit the influence of an outlier on other weights, but they require more scale and zero-point metadata. The notebook's sample matrix shows a largest row maximum 6.7 times the median row maximum, illustrating why one tensor-wide range can spend resolution on an outlier rather than typical rows.

3.2 Packing and metadata overhead

The implementation packs actual low-bit integers into `uint8` storage: two 4-bit values fit in one byte and four 2-bit values fit in one byte. Each quantized row or group also needs a scale stored as FP16. Asymmetric configurations additionally store an 8-bit zero-point. The notebook expresses the approximate effective bit rate for a group of G weights as $b + \frac{24}{G}$ for asymmetric quantization and $b + \frac{16}{G}$ for symmetric quantization. For W4 group-128, the asymmetric metadata adds 0.1875 bits per weight, giving 4.1875 bits per weight in the CSV. Measured model size is calculated from the model's distinct stored tensors; it is not the total size of the exported directory, tokenizer, or notebook.

The embedding table is handled separately: every tested configuration uses 8-bit per-row symmetric quantization for the embedding table. When input and output embeddings are tied, the quantized table is shared rather than stored twice. This is why the experiment comparison changes the transformer-block linear layers while keeping the embedding treatment consistent.

3.3 Error and quality metrics

The notebook reports SQNR as $10 \log_{10}\left(\frac{\sum(w^2)}{\sum((w-\hat{w})^2)}\right)$. A higher SQNR means the reconstructed weights have less squared error relative to the original weight energy. It is a weight-level metric and is not itself a language-model quality score.

The model-level quality metric is perplexity on the WikiText-2 test text. The notebook tokenizes the test split and computes cross-entropy over non-overlapping 512-token windows, then exponentiates the mean token loss. Lower perplexity is better in this experiment. Perplexity can respond differently from average weight error because it measures the model's next-token predictions after quantization, not just distance between each original and reconstructed matrix.

3.4 Clipping search

For the optional clipping variant, the notebook tries a sequence of narrower min/max ranges from 100% down to 70% in 2.5-percentage-point steps. It chooses the range with the lowest squared reconstruction

error for each row. This search improved the recorded weight SQNR for W4 group-128 from 19.34 dB to 19.82 dB, but it did not improve WikiText-2 perplexity: the clipped model recorded 21.04 versus 20.65 without clipping. The comparison is a direct example of why a better local reconstruction score does not guarantee a better end-to-end model score.

4 Project structure and implementation flow

4.1 Main components

The notebook defines a `QConfig` data structure that records bit width, granularity, group size, symmetry, and whether clipping search is active. Seven configurations are collected in `CONFIGS`. The row-layout functions reshape each linear-layer matrix so that a row corresponds to the weights that share a scale. Quantization then calculates codes and metadata for the rows, processes large tensors in bounded chunks, and packs the low-bit codes into bytes.

`QuantLinear` stores packed codes (`qweight`), FP16 scales, any asymmetric zero-points, and the original bias. Its `forward` method unpacks and dequantizes the stored weights and calls PyTorch's linear operation. Therefore, this implementation demonstrates compact weight storage but does not execute a fused integer matrix multiplication. `QuantEmbedding` supplies the dedicated 8-bit embedding path and handles tied embedding/output-head storage.

4.2 End-to-end flow

1. **Setup:** select Qwen/Qwen2.5-0.5B, device and dtype, target size reduction, perplexity tolerance, and evaluation settings.
2. **Define the quantizer:** create configuration records and implement row layouts, min/max ranges, scale and zero-point calculation, clipping search, low-bit packing, and unpacking.
3. **Validate mechanics:** run round-trip, bit-width, granularity, clipping, and linear-forward checks before loading the full evaluation results.
4. **Load model and data:** load the pretrained model and tokenizer and tokenize the WikiText-2 test split.
5. **Convert and measure:** copy the FP16 baseline for each configuration, replace linear layers, quantize embeddings consistently, calculate stored tensor size and SQNR, then measure perplexity, decoding throughput, and peak allocated GPU memory.
6. **Write analysis artifacts:** save `results.csv` and `layer_sqnr.csv`, create the notebook visualizations, and apply the stated two-constraint selection rule.
7. **Export and check:** quantize the selected configuration again, save packed weights plus model/tokenizer configuration, reload through the custom loader, and compare logits before publishing.

4.3 Sanity checks recorded in the notebook

The notebook reports successful pack/unpack round trips for 2-, 4-, and 8-bit values across several tensor lengths, including odd lengths. On synthetic weights, its measured SQNR rises from 5.2 dB at 2 bits to 19.2 dB at 4 bits and 43.8 dB at 8 bits. With a deliberately enlarged outlier row, finer scaling improves SQNR from 8.2 dB per-tensor to 18.7 dB per-channel and 22.3 dB group-wise. A test asserts that the clipping search does not increase reconstruction error on its synthetic case. Finally, `QuantLinear` is checked against a standard

linear operation for all seven configurations with input dimensions 96 and 130; the latter exercises padding when a dimension is not a multiple of the group size.

5 Experimental design

The tested model is Qwen/Qwen2.5-0.5B with 494 million parameters. The notebook reports an FP16 tensor size of 942.3 MB and runs on a Tesla T4 with PyTorch 2.11.0+cu130. The quality baseline is evaluated on the entire tokenized WikiText-2 test split reported as 299,078 tokens. Perplexity uses 512-token windows and batch size 4. Greedy decoding throughput is measured for 128 new tokens and summarized as the median of three runs, after an 8-token warm-up. Peak GPU memory is the CUDA peak allocated-memory reading during measurement.

The two selection thresholds are applied together. The required size reduction is at least 60%, which means the model must be at or below approximately 376.9 MB. The maximum permitted perplexity is the FP16 baseline multiplied by 1.15: approximately 19.77. A configuration that passes only one threshold is not considered to satisfy the project's combined target.

The seven configurations cover W8 symmetric per-tensor and per-channel scaling; W4 asymmetric per-tensor, per-channel, and group-128 scaling; W4 group-128 with clipping search; and W2 asymmetric group-64 scaling. Each run quantizes embeddings using the same W8 per-row scheme so that the main comparison is among the transformer linear-layer configurations.

6 Results from `results.csv`

6.1 Size and quality table

The first table reproduces the result CSV's model-level size and quality measures, rounded for readability. The verdict column compares each result with the 60% size and +15% perplexity limits. `PPL change` is measured against the FP16 baseline.

Configuration	Size MB	Smaller	PPL	PPL change	Target result
FP16 baseline	942.3	0.0%	17.19	0.0%	Reference
W8 per-tensor	471.5	50.0%	18.03	+4.9%	Quality yes; size no
W8 per-channel	472.1	49.9%	17.24	+0.2%	Quality yes; size no
W4 per-tensor	300.9	68.1%	10,446,819	+60,758,366%	Size yes; quality no
W4 per-channel	301.7	68.0%	30.79	+79.1%	Size yes; quality no
W4 group-128	308.9	67.2%	20.65	+20.1%	Size yes; quality no
W4 group-128 +clip	308.9	67.2%	21.04	+22.4%	Size yes; quality no
W2 group-64	231.6	75.4%	163,940	+953,372%	Size yes; quality no

Table 1: Model size, reduction and WikiText-2 perplexity recorded in `results.csv`.

The two W8 configurations retain acceptable perplexity, but their roughly 50% reductions are below the 60% target. Every W4 and W2 configuration crosses the size threshold, but none remains within the +15% quality budget. Thus, the table contains no "both targets met" result.

Among size-qualified configurations, W4 group-128 has the lowest perplexity at 20.65 and is therefore the notebook's fallback choice. It is still above the 19.77 quality ceiling. W4 per-channel is substantially worse at 30.79. Per-tensor W4 collapses to extremely high perplexity, while W2 group-64 also has a very large

quality loss despite producing the smallest model. These results distinguish "smaller" from "usable under the selected quality rule."

6.2 Weight error and runtime table

The second table adds the CSV's effective bits per weight, overall linear-weight SQNR, decoding throughput, and peak allocated GPU memory. The baseline has no quantization SQNR entry in `results.csv`.

Configuration	Bits/weight	SQNR dB	Tokens/s	Peak GPU MB
FP16 baseline	16.00	—	25.4	3,922.7
W8 per-tensor	8.00	25.6	11.9	3,444.1
W8 per-channel	8.01	40.0	13.4	3,444.8
W4 per-tensor	4.00	3.2	10.5	3,273.8
W4 per-channel	4.02	16.3	10.5	3,274.5
W4 group-128	4.19	19.3	10.3	3,281.6
W4 group-128 +clip	4.19	19.8	10.5	3,281.6
W2 group-64	2.38	6.5	9.6	3,208.8

Table 2: Weight-error and runtime measures from `results.csv`; GPU memory is peak allocated memory. Across all quantized runs, recorded decoding speed is lower than FP16's 25.4 tokens per second: the quantized values range from 9.6 to 13.4 tokens per second. This matches the implementation flow: each linear forward pass dequantizes before calling the standard linear operation. The peak allocated GPU-memory column falls from 3,922.7 MB at FP16 to 3,208.8–3,444.8 MB for the quantized configurations. These measurements show lower peak allocation alongside lower model tensor size, but they do not establish a latency benefit.

6.3 Figures from the notebook

The following figures are extracted from the notebook's saved plot outputs and retained as figures rather than redrawn.

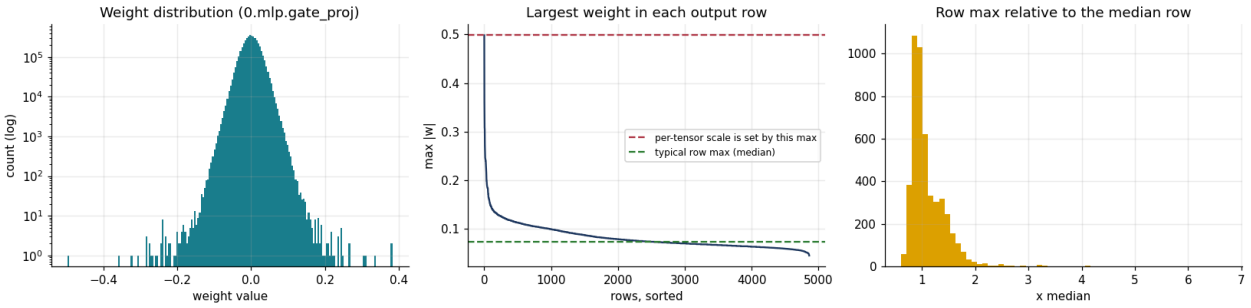


Figure 1: Exploratory weight analysis: value distribution, sorted maximum absolute value per output row, and row maxima relative to their median for one first-block MLP projection matrix. The notebook reports a 6.7x largest-to-median row-maximum ratio.

This probe explains the sensitivity to scale granularity. A single per-tensor range must accommodate the largest row maximum, even though the other rows are typically smaller. Per-row or per-group ranges localize the effect of that range choice.

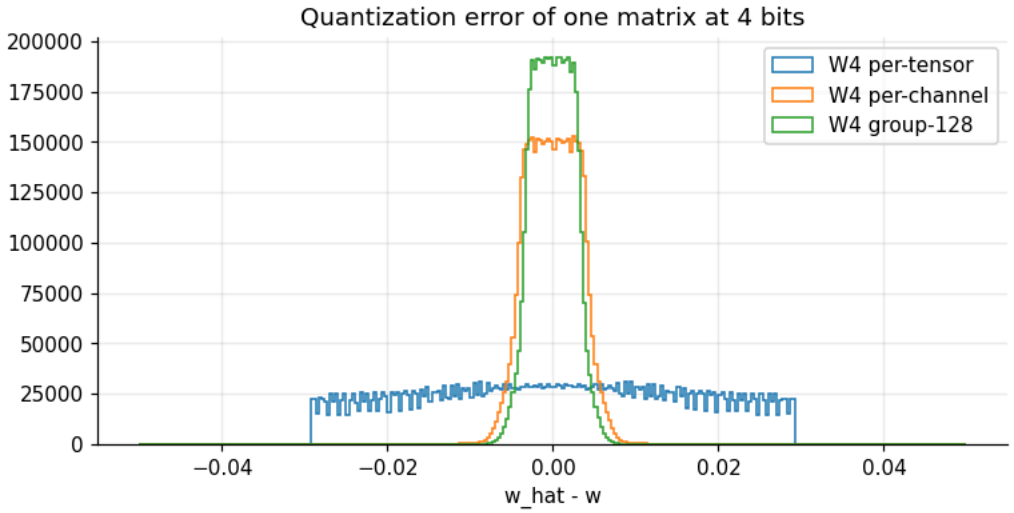


Figure 2: Reconstruction-error distributions for three W4 schemes on the notebook's sample matrix.

The error-distribution plot compares per-tensor, per-channel and group-128 W4. The corresponding sample-matrix SQNR in the notebook is 3.28 dB, 17.55 dB and 19.72 dB, respectively. This is one matrix-level probe; the overall model-weight and perplexity results are summarized separately in the CSV tables.

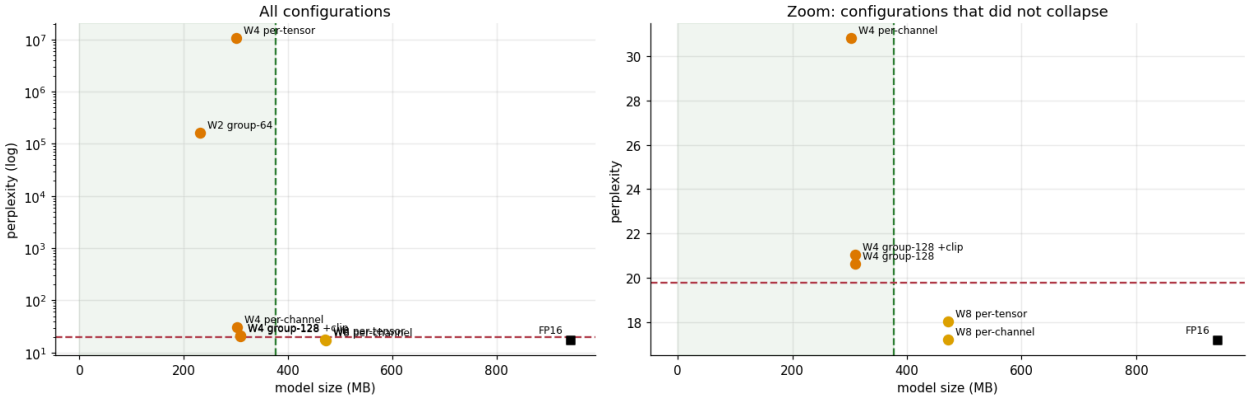


Figure 3: Notebook trade-off chart: model size against perplexity. The shaded size region marks the required size range and the horizontal reference marks the quality limit; the right panel zooms in on configurations that did not collapse.

This is the direct visual test of the project rule: a valid result would need to fall both within the size region and below the perplexity line. The CSV values confirm that none of the seven tested points satisfies both conditions. The W8 points preserve perplexity but sit beyond the size limit; the size-qualified W4 and W2 points lie above the quality limit.

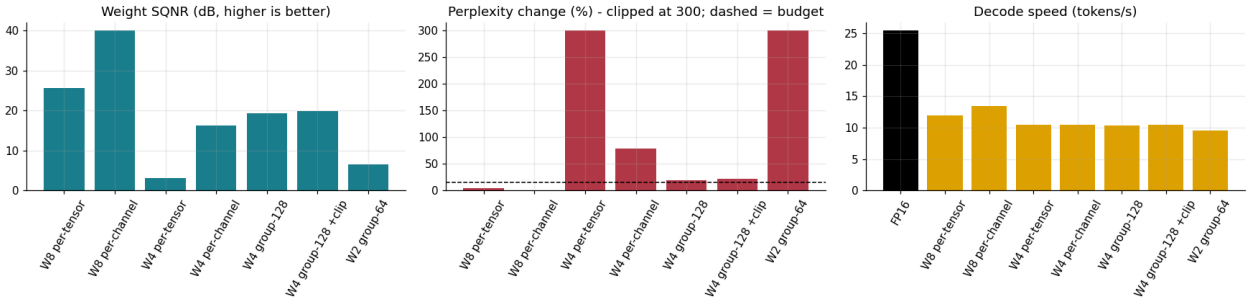


Figure 4: Notebook comparison of overall weight SQNR, perplexity change (display clipped at 300%), and decoding speed. The dashed line in the middle panel is the +15% quality budget.

The visual comparison makes clear that the extremely large perplexity changes for per-tensor W4 and group-64 W2 exceed the plotted cap. It also shows that high SQNR does not guarantee the lowest perplexity: clipping raises W4 group-128 SQNR from 19.3 to 19.8 dB, while the clipped configuration's PPL is slightly higher, 21.04 rather than 20.65.

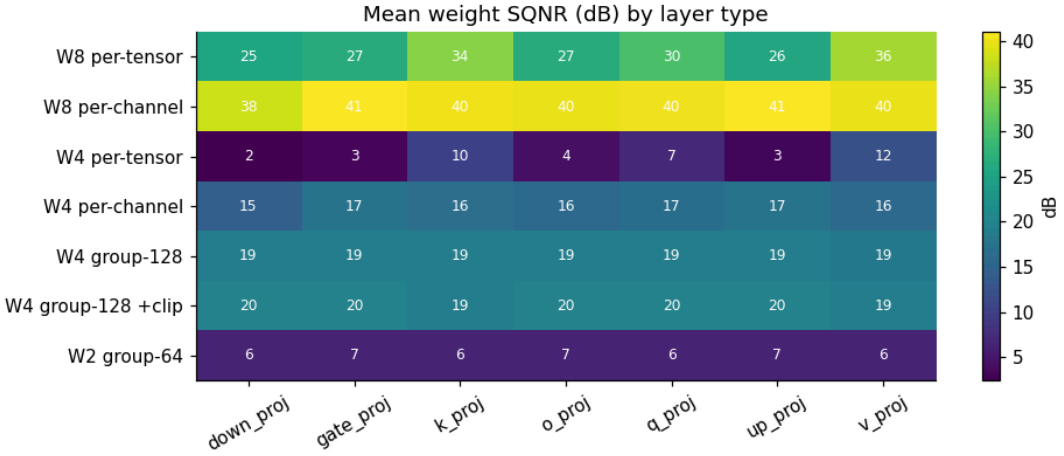


Figure 5: Notebook heatmap of mean layer SQNR by quantizer configuration and linear-layer type, computed from `layer_sqnr.csv`.

The layer-type means show the broad improvement from finer scaling. For W4 per-channel, mean SQNR across layer types ranges from 14.61 dB (`down_proj`) to 17.21 dB (`up_proj`). W4 group-128 is more even in this CSV, ranging from 18.64 dB (`v_proj`) to 19.46 dB (`up_proj`). W8 per-channel ranges from 38.20 to 41.02 dB. These group means support the overall result that group-wise ranges reduce weight reconstruction error compared with W4 per-channel scaling in this run.

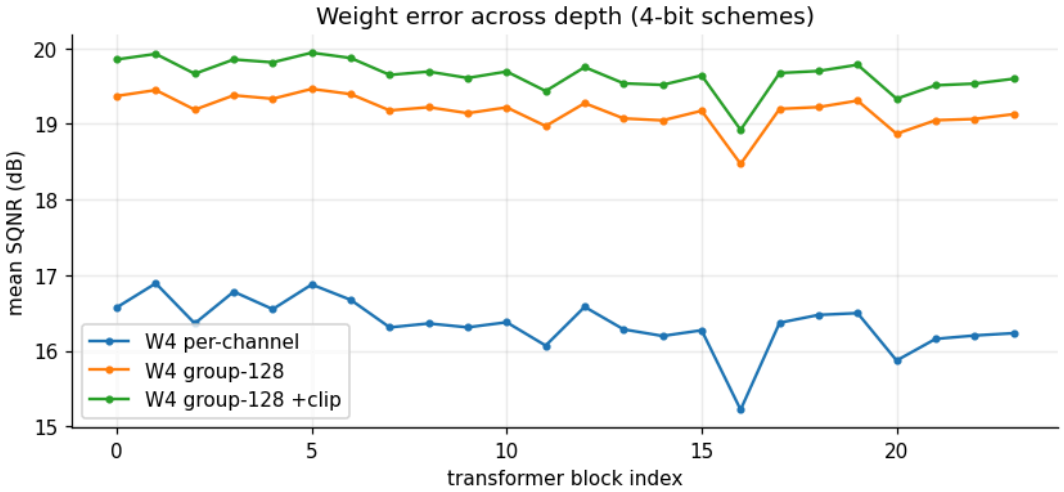


Figure 6: Notebook plot of mean SQNR over transformer block depth for three W4 configurations.

The depth chart offers a layer-by-layer view instead of only a model-wide average. It compares W4 per-channel, W4 group-128, and W4 group-128 with clipping across transformer block indices. The group-wise curves sit above the per-channel curve in the figure, while clipping remains a modest change in the layer-weight metric; its end-to-end perplexity is not correspondingly better.

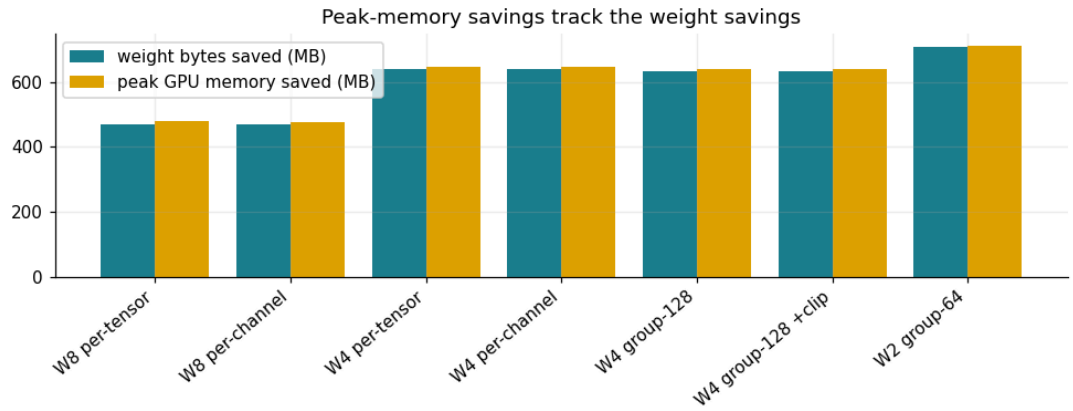


Figure 7: Notebook comparison of weight bytes saved and peak GPU memory saved relative to FP16. The memory chart places stored weight savings beside reductions in peak allocated GPU memory for each configuration. Both quantities decline as the configurations become smaller in the recorded run. Runtime is not improved, so memory reduction and decode acceleration should not be treated as the same outcome for this implementation.

7 Interpretation and project conclusion

The strongest result is the joint effect of bit width and scale granularity. Reducing to W4 saves enough storage to clear the project's size requirement, but a whole-tensor scale performs disastrously on the tested model. Per-channel scaling reduces that failure substantially, and group-128 scaling improves W4 further while adding metadata: its effective rate is 4.19 rather than exactly 4 bits per weight. Group-128 is the best of the size-qualified runs by perplexity, but its +20.1% change still exceeds the allowed +15%.

The W8 runs define the other side of the trade-off. They remain within the quality limit—especially W8 per-channel at +0.2%—but their roughly 50% reductions are insufficient for a 60% target. Based strictly on these seven experiments, the target combination is unresolved rather than achieved. If the priority is preserving the defined quality threshold, W8 per-channel is the closest measured option but misses size; if the priority is satisfying size, W4 group-128 is the best fallback but misses quality. Which compromise is acceptable depends on the project's priority, but the experiment's stated rule requires both.

The measurements also show why weight SQNR must be read with perplexity. Group-wise scaling tends to improve reconstruction metrics, and clipping slightly improves overall SQNR in this case, but the clipped model has worse perplexity than the unclipped group-128 model. Similarly, the smallest W2 result has the lowest storage size and one of the lowest SQNR values, but its perplexity is far outside the quality budget. These outputs support evaluating the actual model task in addition to inspecting local weight error.

The selected W4 group-128 model was written in a custom packed format and reloaded using the notebook's custom loader. The notebook verifies the reload by comparing logits on random token IDs and reports a maximum absolute difference of 5.86e-2, just under its 6e-2 assertion threshold. The notebook also states that standard `from_pretrained` loading is not supported for this custom packed format; its loader reconstructs the architecture, installs quantized layer shells, and loads the saved packed tensors.

7.1 Scope of the evidence

The conclusions are limited to the one Qwen/Qwen2.5-0.5B model, the seven listed quantization configurations, the notebook's WikiText-2 perplexity setup, and its single-prompt greedy decoding benchmark

on a Tesla T4. The CSVs provide recorded point values rather than repeated-run variability or uncertainty estimates. The project therefore establishes the trade-offs observed in this notebook, but does not establish that the same ranking or thresholds will hold for different models, datasets, hardware, or evaluation procedures.

The notebook itself lists possible next experiments: smaller groups, mixed precision that retains higher precision in sensitive layers, calibration-based rounding, and broader evaluation. These are future directions noted by the project, not results of the current CSVs.

8 Glossary

perplexity – **perplexity**: The exponential of mean token-level cross-entropy in the notebook's WikiText-2 evaluation; lower is better.

quantization – **weight quantization**: Representing model weights with a smaller set of integer values and the scale metadata needed to reconstruct approximations.

scale – **quantization scale**: The step size that maps between an integer level and its approximate floating-point weight.

SQNR – **signal-to-quantization-noise ratio**: A weight-reconstruction measure computed as 10 log10 of signal energy divided by squared quantization error; higher is better.

zero-point – **zero-point**: The integer offset used by the notebook's asymmetric quantizer to represent values around zero.

9 Project links

The project repository and the published custom-quantized model are linked below. The second supplied URL is a Hugging Face model page, so it is labeled as Hugging Face rather than HubSpot.

> **GitHub repository:** https://github.com/eltadlll/Quantizer_from_scratch
> **Hugging Face model:** https://huggingface.co/EliasTad/qwen2.5-0.5b-w4-custom-quant